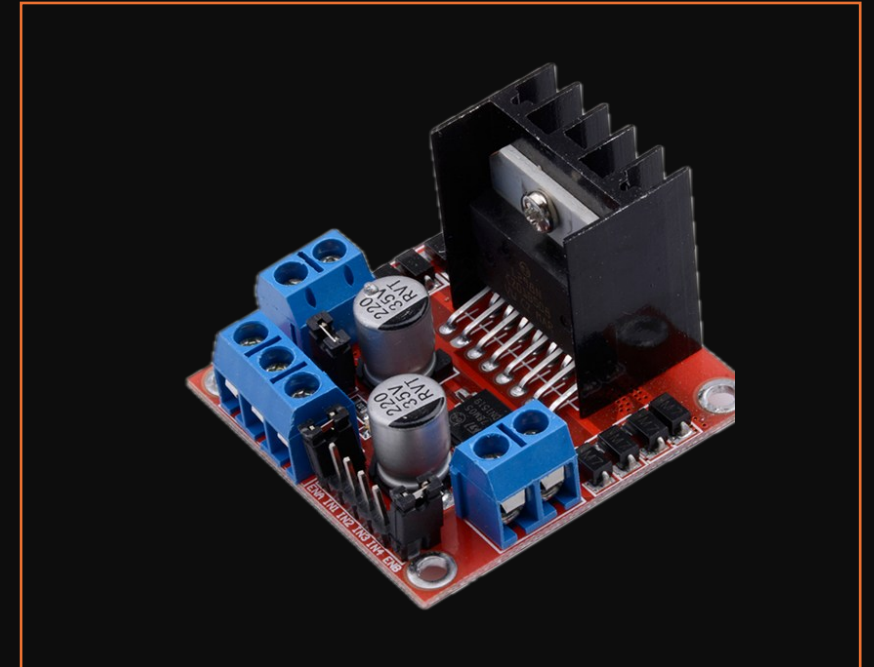


Quantum Magnetometry Using Helmholtz Coils and Arduino-Controlled Currents

Rishabh Venkataramani

Arduino Setup

- We had an Arduino Uno that controls two L298n dual h-bridge motor drivers. Driver 1 controls the Z direction and the motor, while Driver 2 controls the X and Y directions
- A motor can be used to test to see if the setup is working (magnetic field will be produced)
- When uploading the sketch, each time the correct port must be chosen



Arduino Sketches

- We had many different sketches to test out the setup
 - allConstant: A constant PWM (0-255) waveform to all 3 axes and the motor
 - allSinusoidal: Periodic 0.2 Hz PWM waveform to all 3 axes and the motor
 - motorConstant: A constant PWM (0-255) waveform to only the motor
 - motorSinusoidal: Periodic 0.2 Hz PWM waveform to only the motor
 - stopCode: PWM set to 0 to stop current flow
-

```
pinMode(IN1_X, OUTPUT); pinMode(IN2_X, OUTPUT); pinMode(EN_X, OUTPUT);  
pinMode(IN1_Y, OUTPUT); pinMode(IN2_Y, OUTPUT); pinMode(EN_Y, OUTPUT);  
pinMode(IN1_MOTOR, OUTPUT); pinMode(IN2_MOTOR, OUTPUT); pinMode(EN_MOTOR, OUTPUT);  
pinMode(IN1_Z, OUTPUT); pinMode(IN2_Z, OUTPUT); pinMode(EN_Z, OUTPUT);
```

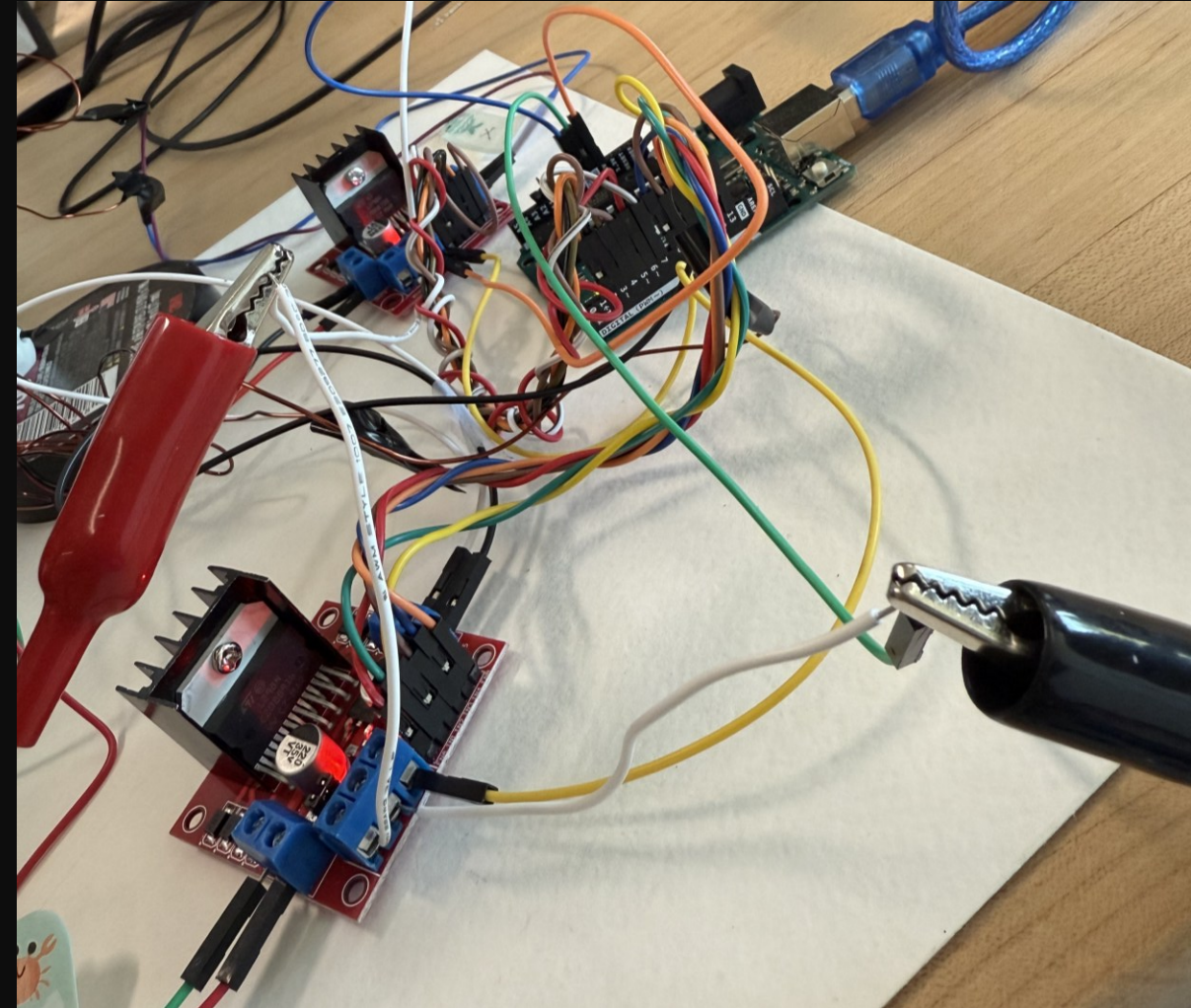
```
// Set direction for all outputs (forward)  
digitalWrite(IN1_X, HIGH); digitalWrite(IN2_X, LOW);  
digitalWrite(IN1_Y, HIGH); digitalWrite(IN2_Y, LOW);  
digitalWrite(IN1_Z, HIGH); digitalWrite(IN2_Z, LOW);  
digitalWrite(IN1_MOTOR, HIGH); digitalWrite(IN2_MOTOR, LOW);
```

```
int constantPWM = 255;
```

```
analogWrite(EN_X, constantPWM);  
analogWrite(EN_Y, constantPWM);  
analogWrite(EN_Z, constantPWM);  
analogWrite(EN_MOTOR, constantPWM);
```

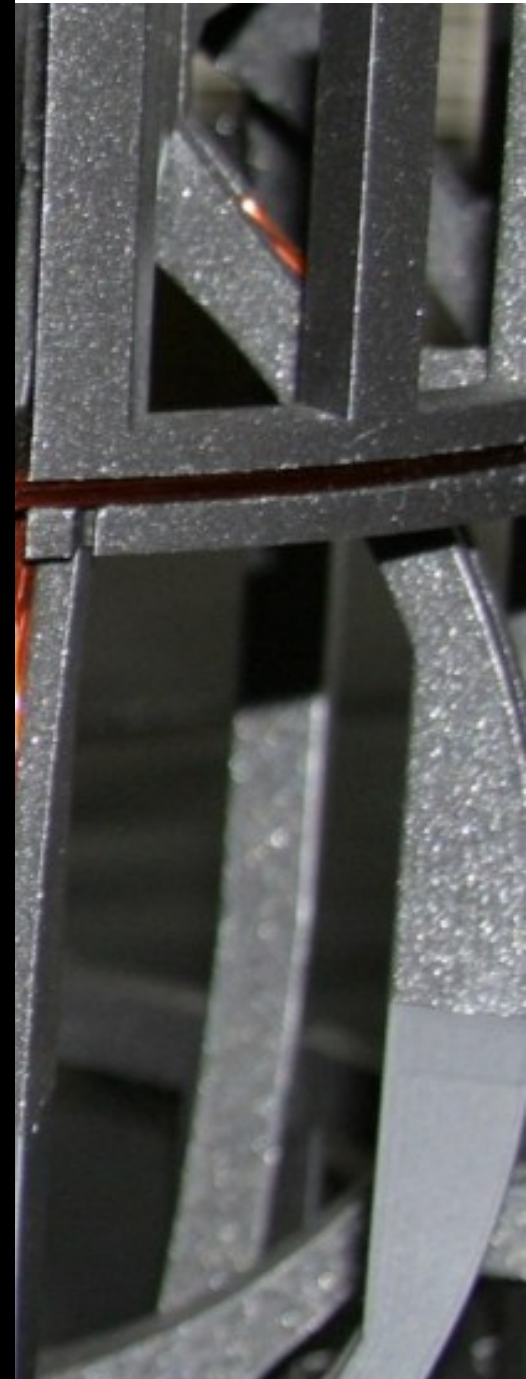
Wiring

- Trouble with powering the circuit
 - Batteries ran out of charge quickly, making it not a good source of power
 - We used a DC power source outputting 7 volts and 1 ampere
 - The grounds of the L298's were connected to the DC ground through alligator clips, and the same was done for the power
 - Testing by running the code, the motor started running
-



Helmholtz Coil

- The Helmholtz coil system consists of three orthogonal pairs of coils generating magnetic field at the center of the system along all three axes (X, Y, and Z)
 - Allows for consistent measurements
- Each coil pair has 10 turns of wire with radius approximately 17.8 centimeters
- From Biot-Savart's law ($B = \left(\frac{4}{5}\right)^{3/2} \frac{\mu_0 NI}{R}$), the magnetic field generated at the center of the Helmholtz coil by the coil pairs is $(5.05 \times 10^{-5})I$ teslas

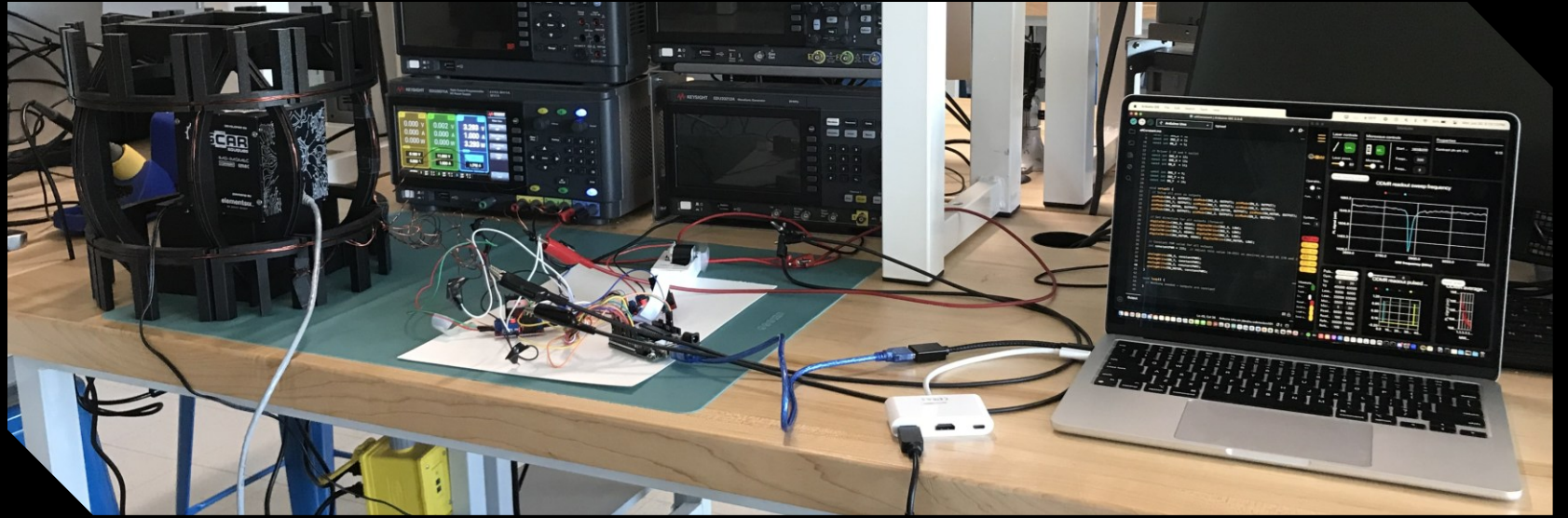


Quantum Magnetometer

- This is a diamond nitrogen-vacancy (NV) center-based magnetometer
 - The defect responds to magnetic fields through changes in the spin state, allowing for precise measurements
- Initially, we had trouble running the EduQube software necessary to obtain the measurements.
 - The zip file download contained the contents for the application
 - We had to move all the contents into a new folder and rename it to EduQube.app
 - macOS was able to recognize the application bundle, and the software launched
- We tested out measurements via a magnet to see if we were able to see the correct number of dips for the distance between the magnet and the magnetometer



Full Setup



1. Sketch is compiled on the computer and uploaded to Arduino
2. Current starts running based on the values specified in the sketch
3. Magnetic field is produced at the center of the Helmholtz coil due to the current in the wire loops
4. The magnetic field is detected by the magnetometer and saved as a .csv where the center frequencies can be extracted

Unfortunately, no splitting was detected when uploading the sketch, which is probably either due to the current being too low in the coils, or there being a wiring error.

Reflection

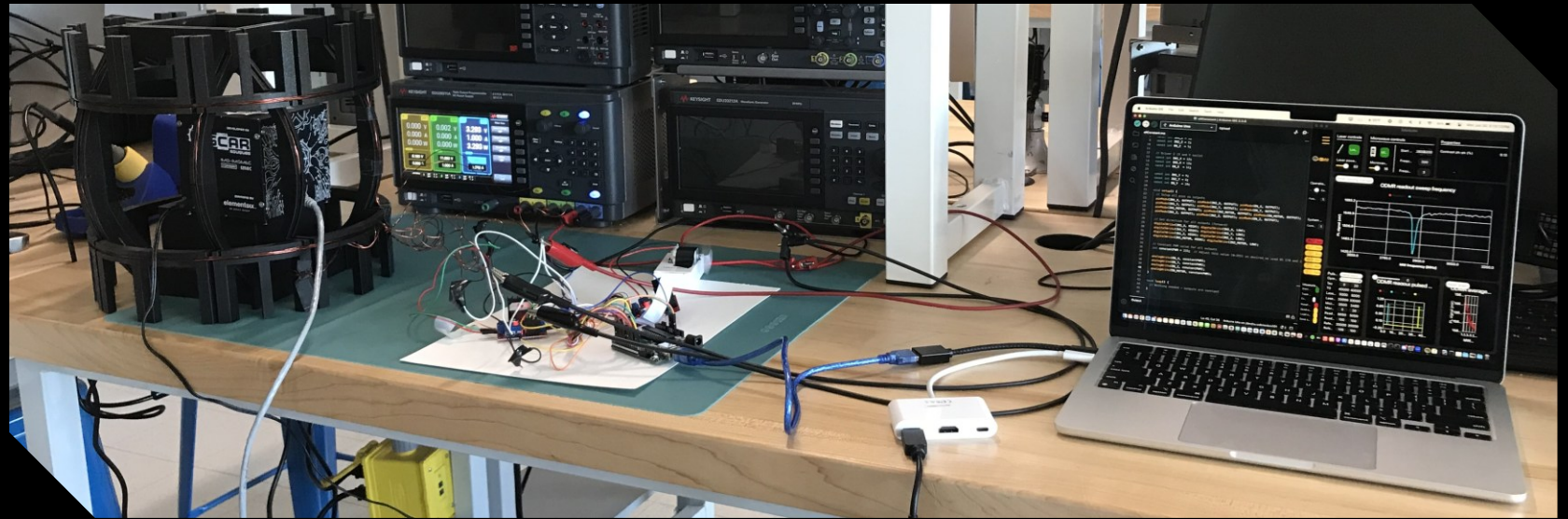
Throughout the Quantum Explorers Program, I gained much hands-on experience with Arduino, learning how to wire it up to a power source to control current. I also developed an approach to debugging step by step to identify and resolve issues in our experimental setup. Additionally, I learned how to package the contents of an application so that it can be recognized and executed by macOS.

I am especially grateful to Professor Marina Radulaski and Elnaz Hamdarsi for giving me this opportunity to work at their lab, as it was an inspiring experience.

Quantum Magnetometer: Browser Based Control for Arduino Uno and Helmholtz Coil Setup

Rishabh Venkataramani

Full Setup



1. Sketch is compiled on the computer and uploaded to Arduino
2. Current starts running based on the values specified in the sketch
3. Magnetic field is produced at the center of the Helmholtz coil due to the current in the wire loops
4. The magnetic field is detected by the magnetometer and saved as a .csv where the center frequencies can be extracted

Unfortunately, no splitting was detected when uploading the sketch, which is probably either due to the current being too low in the coils, or there being a wiring error.

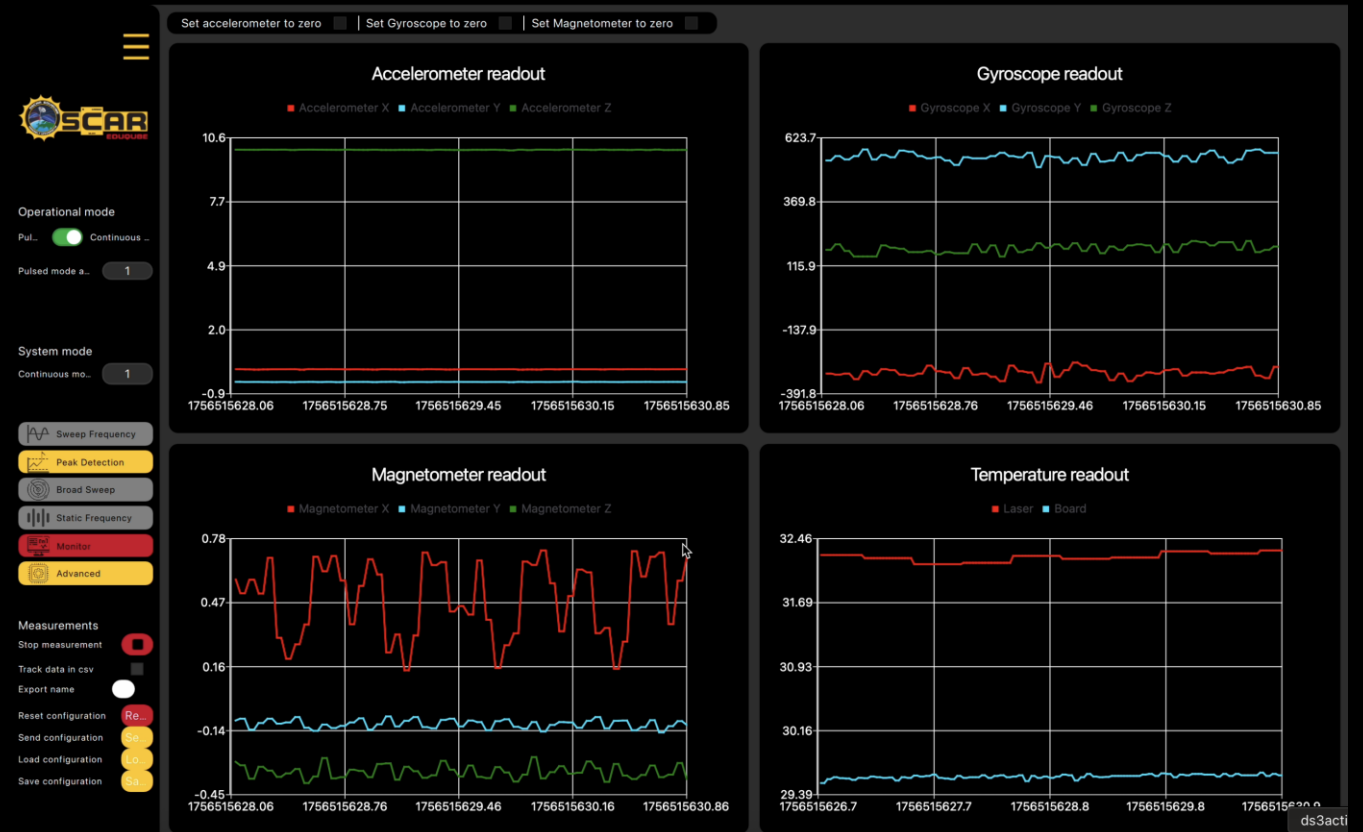
Slightly Tweaked Setup

Wiring was a bit more optimized,
with us having to use less alligator
clips.



Magnetic Field/ Splitting Observed

- We had to select a different view, the Monitor tab was necessary in order to view the minute differences in the magnetometer readout.
- Increased PWM values resulted in steadier magnetic field readouts and a stronger field
 - Smaller values resulted in more fluctuations, which confirms earlier experimentation



Core Features

Helmholtz + Motor (USB via Web Serial)

Use Chrome/Edge on desktop. If file mode blocks serial, serve locally with: `python3 -m http.server` and open `http://localhost:8000/index_webserial_motor.html`.

Helmholtz Coils

X:	Magnitude (0–255)	0	Direction	+
Y:	Magnitude (0–255)	0	Direction	+
Z:	Magnitude (0–255)	0	Direction	+

Motor

Speed (PWM 0–255)	0	Direction	Forward
-------------------	--------------------------------	-----------	--

Protocol sent to Arduino (one line per click):
XMAG=<0–255>, XDIR=<±1>, YMAG=<0–255>, YDIR=<±1>, ZMAG=<0–255>, ZDIR=<±1>, MMAG=<0–255>, MDIR=<±1>

Log

- Magnitude (0–255) + Direction (Forward/Reverse) for X, Y, Z coils
- Motor speed and direction control
- Buttons for Connect, Disconnect, Send, Stop All
- Log panel for transmitted and received messages

Code Structure (HTML)

- `<fieldset>` groups controls (Helmholtz coils, motor)
 - `<input type='number'>` for magnitudes
 - `<select>` dropdowns for directions
 - `<button>` for serial connection and control
 - `<div id='log'>` for output console
-

```
body {
  font-family: "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
  margin: 2rem auto;
  max-width: 800px;
  line-height: 1.6;
  background: #f9fafb;
  color: #222;
}

h1 {
  margin-top: 0;
  font-size: 1.8rem;
  text-align: center;
  color: #2a4d8f;
}

fieldset {
  border: none;
  border-radius: 12px;
  padding: 1.2rem;
  margin-bottom: 1.5rem;
  background: #fff;
  box-shadow: 0 2px 6px rgba(0,0,0,0.08);
}

legend {
  font-weight: bold;
  color: #2a4d8f;
  padding: 0 .4rem;
}
```

Code Structure (CSS)

- Shadows, rounded corners, and modern color scheme
 - Improved readability and professional look
 - Example: button styling with padding, radius, colors
-

```
<style>
.row {
  align-items: center;
  margin: .6rem 0;
  flex-wrap: wrap;
}

.buttons {
  display: flex;
  flex-wrap: wrap;
  gap: .8rem;
  justify-content: center;
  margin: 1.2rem 0;
}

button {
  padding: .6rem 1.2rem;
  border-radius: 8px;
  border: none;
  cursor: pointer;
  font-size: .95rem;
  font-weight: 500;
  transition: background 0.2s ease, transform 0.1s ease;
  background: #2a4d8f;
  color: #fff;
}
```

Code Structure (JavaScript)

- Connect/Disconnect: open and close Web Serial port
 - sendOnce(): reads inputs, clamps values (0–255), builds command string
 - Command format: XMAG, XDIR, YMAG, YDIR, ZMAG, ZDIR, MMAG, MDIR
 - Log function updates console for debugging
-

```
function clamp255(n) {  
  n = Number(n) || 0;  
  if (n < 0) n = 0;  
  if (n > 255) n = 255;  
  return Math.round(n);  
}
```

```
async function sendOnce() {  
  if (!writer) return;  
  const xmag = clamp255($("#xmag").value);  
  const xdir = Number($("#xdir").value) || 1;  
  const ymag = clamp255($("#ymag").value);  
  const ydir = Number($("#ydir").value) || 1;  
  const zmag = clamp255($("#zmag").value);  
  const zdir = Number($("#zdir").value) || 1;  
  const mmag = clamp255($("#mmag").value);  
  const mdir = Number($("#mdir").value) || 1;
```

Styling and Compatibility Changes

- Clean styling with HTML + CSS (modern fonts, shadows, consistent buttons)
 - Controls aligned in one line: Magnitude + Direction together
 - Inputs capped at 255 for safety
 - Removed unnecessary helper texts for simplicity
 - No special software installation needed
 - Bluetooth not necessary
 - Works on any computer with Chrome/Edge + USB Web Serial
 - Portable: open HTML file, plug in Arduino, start controlling coils
-

Helmholtz + Motor Control

Helmholtz Coils

X: Magnitude (0–255):

Direction:

Forwa ▾

Y: Magnitude (0–255):

Direction:

Forwa ▾

Z: Magnitude (0–255):

Direction:

Forwa ▾

Motor

Motor: Speed (0–255):

Direction:

Forwa ▾

Connect

Disconnect

Send

Stop All

Log

Future Work

- Add presets (e.g., Uniform +X, Rotate field)
- Control the current for sinusoidal PWM
- Add in Magnetometer Control
 - Add 3D visualization of net magnetic field vector
 - Automate sweeps + data logging to CSV